DEEP LEARNING

Lecture 01

OBJECTIVES

- Intro to DL frameworks
- Useful definitions
- Quick PyTorch overview

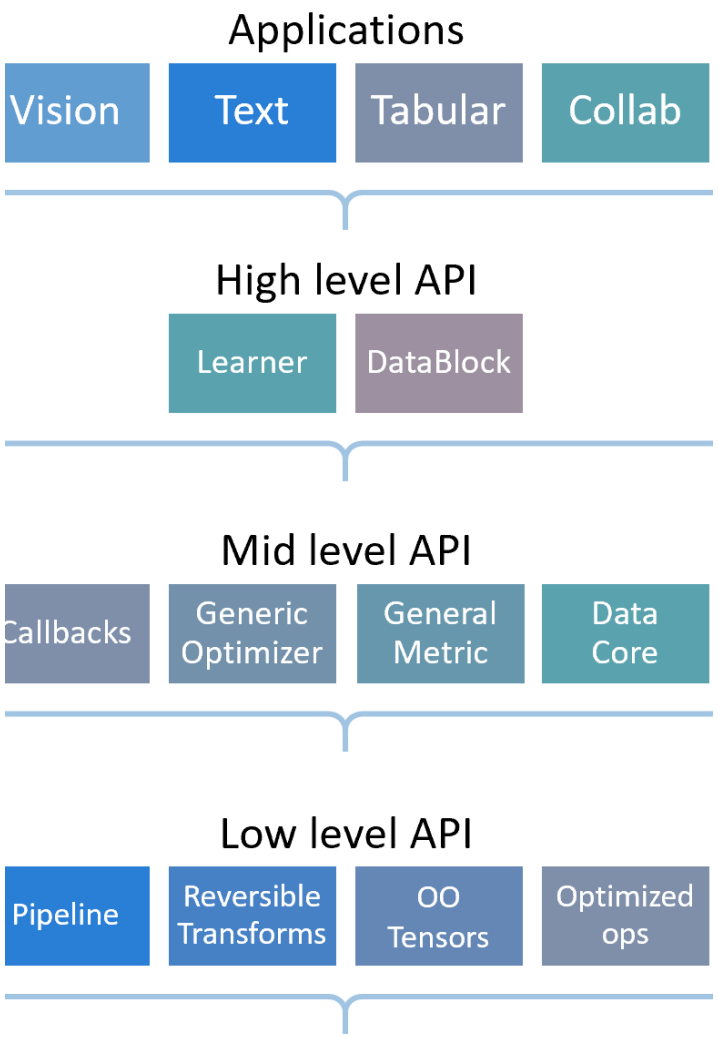
COURSE ORGANIZATION

Examination	Weight	Total
In-class Quiz (5)	2	10%
Assignments (10)	4	40%
Midterm (1)	30	30%
Project (1)	20	20%

- **TensorFlow** (TF) is an end-to-end machine learning framework from Google that allows you to perform an extremely wide range of downstream tasks. With TF2.0 and newer versions, more efficiency and convenience was brought to the game.
- **Keras** is built on top of TensorFlow, which makes it a wrapper for deep learning purposes. It is incredibly user-friendly and easy to pick up. A solid asset is its neural network block modularity and the fact that it is written in Python, which makes it easy to debug.
- **PyTorch** is TensorFlow's direct competitor developed by Facebook, and is widely used in research projects. It allows almost unlimited customization and is well adapted to running tensor operations on GPUs (actually, so is TensorFlow).
- **Scikit-learn** is another user-friendly framework that contains a great variety of useful tools: classification, regression and clustering models, as well a preprocessing, dimensionality reduction and evaluation tools.

- NLTK is a leading platform for building Python programs to work with human language data. It provides easy-to-use interfaces to over 50 corpora and lexical resources such as WordNet, along with a suite of text processing libraries for classification, tokenization, stemming, tagging, parsing, and semantic reasoning, wrappers for industrial-strength NLP libraries, and an active discussion forum.

- fastai is a deep learning library which provides practitioners with high-level components that can quickly and easily provide state-of-the-art results in standard deep learning domains, and provides researchers with low-level components that can be mixed and matched to build new approaches. It aims to do both things without substantial compromises in ease of use, flexibility, or performance. This is possible thanks to a carefully layered architecture, which expresses common underlying patterns of many deep learning and data processing techniques in terms of decoupled abstractions. These abstractions can be expressed concisely and clearly by leveraging the dynamism of the underlying Python language and the flexibility of the PyTorch library



ESSENTIAL TAKEAWAYS

- **The framework and models you end up using to perform a task should depend on that task's nature**
  - If you want to predict people's opinion in movie reviews, then a deep learning approach using Keras or PyTorch makes sense, and if you want to predict the price of future NBA game tickets, then scikit-learn's ability to crunch structured data is all you need.
  - If you are into academic research and want to dive into complex modeling, then TF and PyTorch are made for you, go for it! In contrast, building minimalist yet sturdy architectures codable in 5 minutes, trainable in 2 minutes and deployable in 10 minutes is possible with Keras and scikit-learn.

- **Weigh your options** before committing to learning a machine learning framework in depth
- **Be aware of your learning curve**

KERAS

- Keras is an effective high-level neural network Application Programming Interface (API) written in Python. This open-source neural network library is designed to provide fast experimentation with deep neural networks, and it can run on top of CNTK, TensorFlow, and Theano.
- Keras focuses on being modular, user-friendly, and extensible. It doesn't handle low-level computations; instead, it hands them off to another library called the Backend.
- Keras was adopted and integrated into TensorFlow in mid-2017. Users can access it via the tf.keras module. However, the Keras library can still operate separately and independently.

PYTORCH

- PyTorch is a relatively new deep learning framework based on Torch. Developed by Facebook's AI research group and open-sourced on GitHub in 2017, it's used for natural language processing applications. PyTorch has a reputation for simplicity, ease of use, flexibility, efficient memory usage, and dynamic computational graphs. It also feels native, making coding more manageable and increasing processing speed.

TENSORFLOW

- TensorFlow is an end-to-end open-source deep learning framework developed by Google and released in 2015. It is known for documentation and training support, scalable production and deployment options, multiple abstraction levels, and support for different platforms, such as Android.
- TensorFlow is a symbolic math library used for neural networks and is best suited for dataflow programming across a range of tasks. It offers multiple abstraction levels for building and training models.

- http://www.d2l.ai/chapter_installation/index.html

- pip install torch torchvision
- pip install tensorflow tensorflow-probability

- pip install d2l==1.0.0b0

- *conda install -c conda-forge unzip*
- conda install -c conda-forge m2-unzip

- curl https://d2l.ai/d2l-en.zip -o d2l-en.zip
- unzip d2l-en.zip && rm d2l-en.zip (unzip d2l-en.zip && del d2l-en.zip)
- cd pytorch

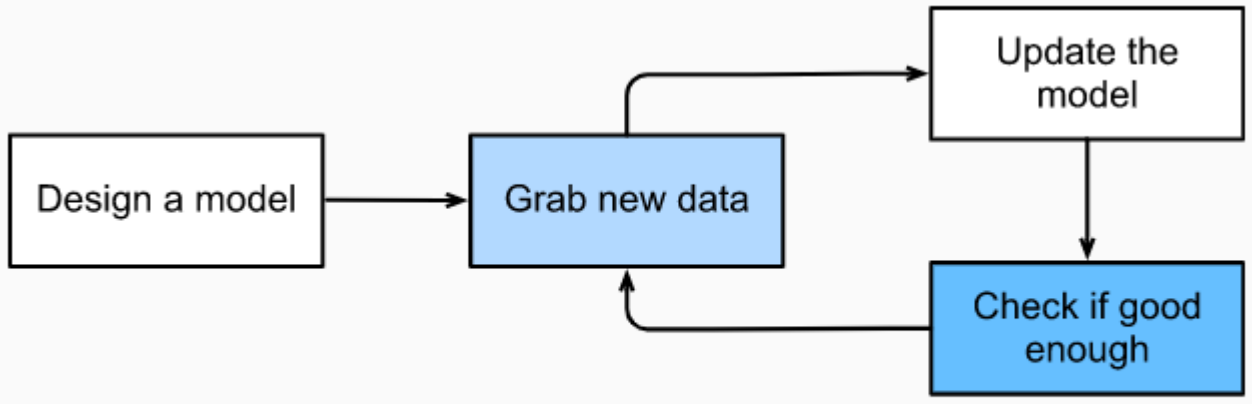


Fig. 1.1.2 A typical training process.

1. The *data* that we can learn from.
2. A *model* of how to transform the data.
3. An *objective function* that quantifies how well (or badly) the model is doing.
4. An *algorithm* to adjust the model's parameters to optimize the objective function.

DATA

MODEL

OBJECTIVE FUNCTIONS

ALGORITHMS

I'LL KEEP BOTH ON TRACK

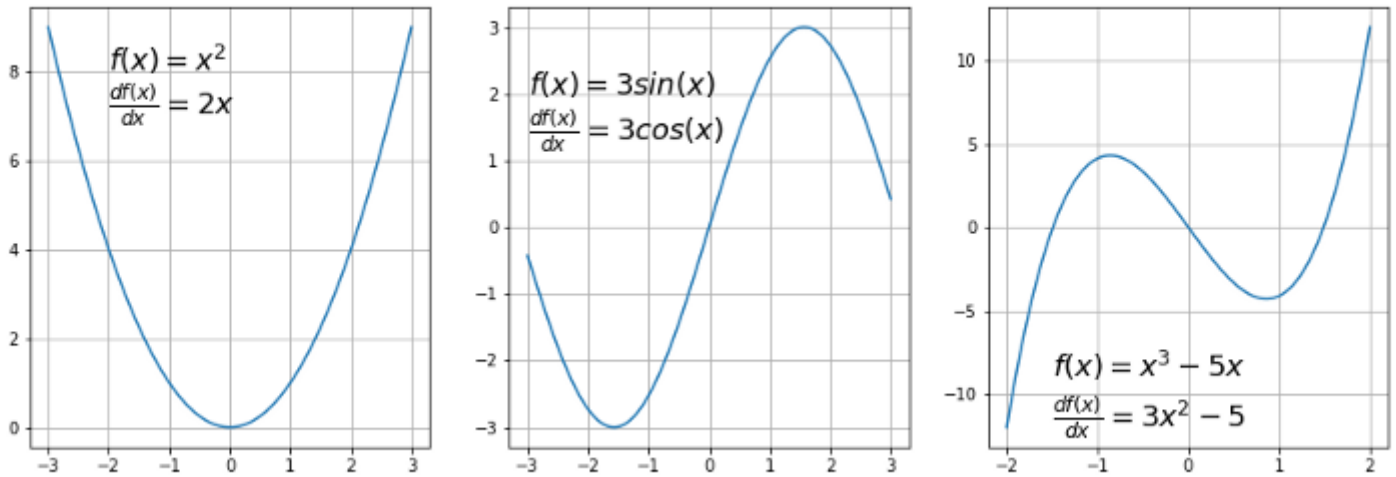
- **What speaks for PyTorch**
  - PyTorch is easier to learn and work with
  - It is better for some projects, especially for NLP and building rapid prototypes
  - New resources come in at a rapid pace, offering PyTorch to be suited well for production environments
  - Most research is done in PyTorch
- **What speaks for TensorFlow**
  - TF's documentation is on point and TF offers large community support
  - TF is currently better for production models and scalability because it was built to be production ready
  - With TF 2.x it basically got most of the Features PyTorch has
  - tf.keras offers a high-level API for fast model

GRADIENT DESCENDENT

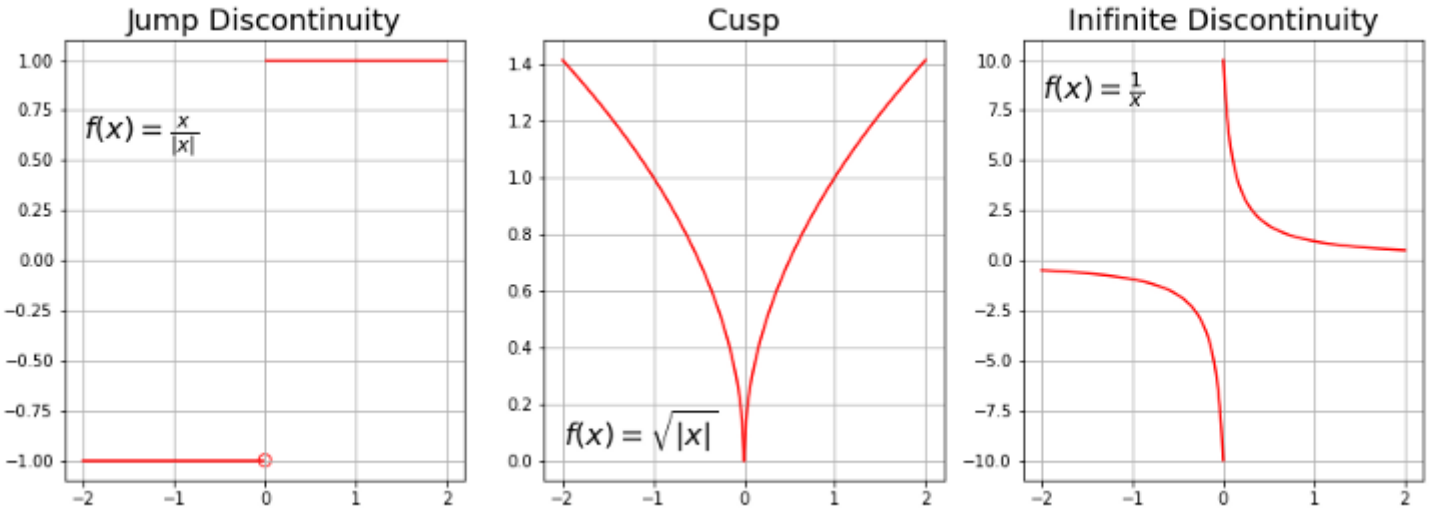
- Gradient descent (**GD**) is an iterative first-order optimization algorithm used to find a local minimum/maximum of a given function. This method is commonly used in machine learning (ML) and deep learning (DL) to minimize a cost/loss function (e.g. in a linear regression)

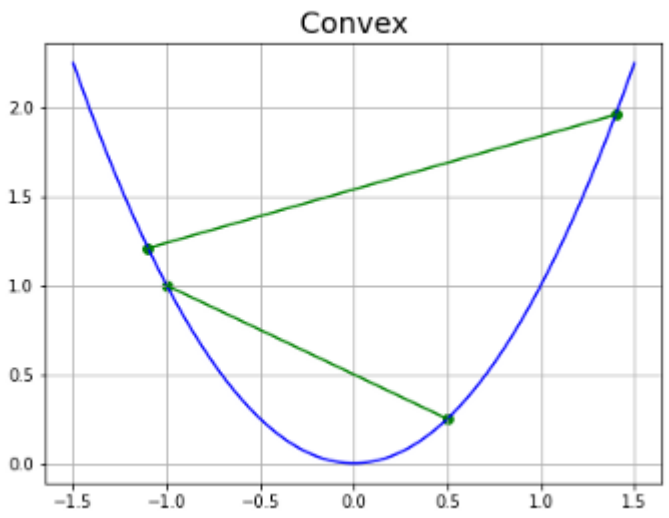
- Gradient descent algorithm does not work for all functions. There are two specific requirements. A function has to be:
  - Differentiable
  - Convex

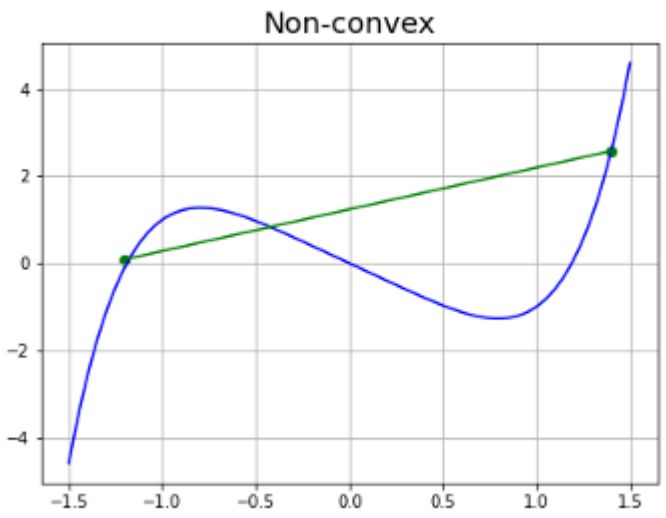
- If a function is differentiable it has a derivative for each point in its domain



- Typical non-differentiable functions have a step a cusp or a discontinuity







- Or

$$\frac{d^2 f(x)}{dx^2} > 0$$

$$f(x) = x^2 - x + 3$$

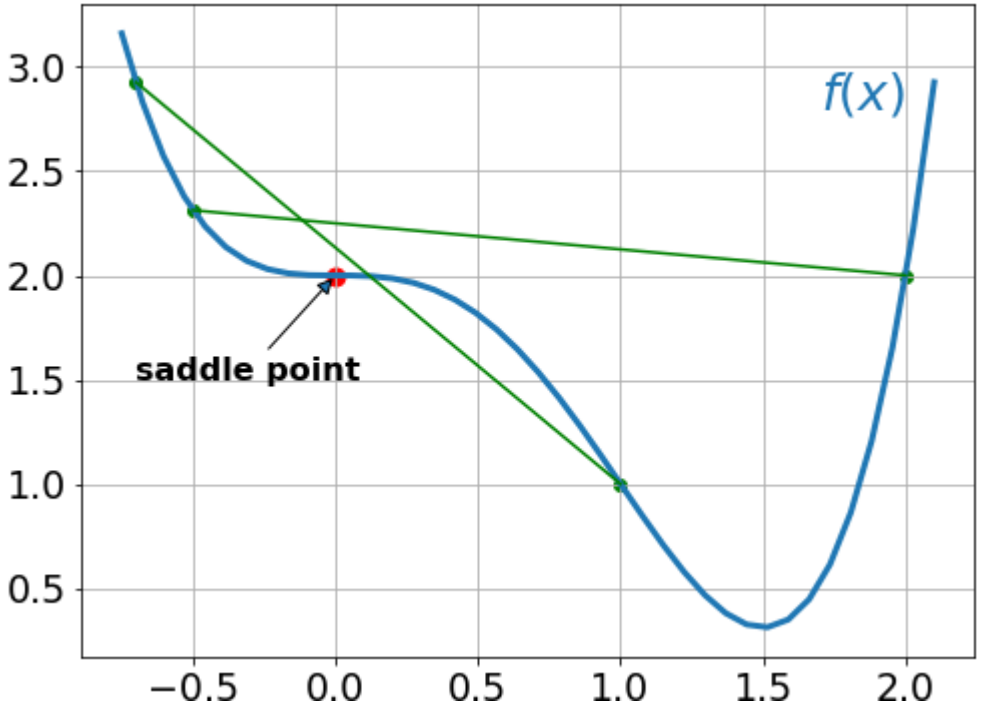
- First and second?

$$\frac{df(x)}{dx} = 2x - 1, \qquad \frac{d^2 f(x)}{dx^2} = 2$$

$$f(x) = x^4 - 2x^3 + 2$$

$$\frac{df(x)}{dx} = 4x^3 - 6x^2 = x^2(4x - 6)$$

$$\frac{d^2 f(x)}{dx^2} = 12x^2 + 12x = 12x(x - 1)$$



- for x<0: function is convex
- for 0<x<1: function is concave (the 2nd derivative < 0)
- for x>1: function is convex again

GD

- Gradient is a slope of a curve at a given point in a specified direction.
- In the case of a **univariate** function, it is simply the **first** derivative at a selected point. In the case of a **multivariate** function, it is a **vector of derivatives** in each main direction (along variable axes). Because we are interested only in a slope along one axis and we don't care about others these derivatives are called **partial derivatives**.

$$\nabla f(p) = \begin{bmatrix} \frac{\partial f}{\partial x_1}(p) \\ \vdots \\ \frac{\partial f}{\partial x_n}(p) \end{bmatrix}$$

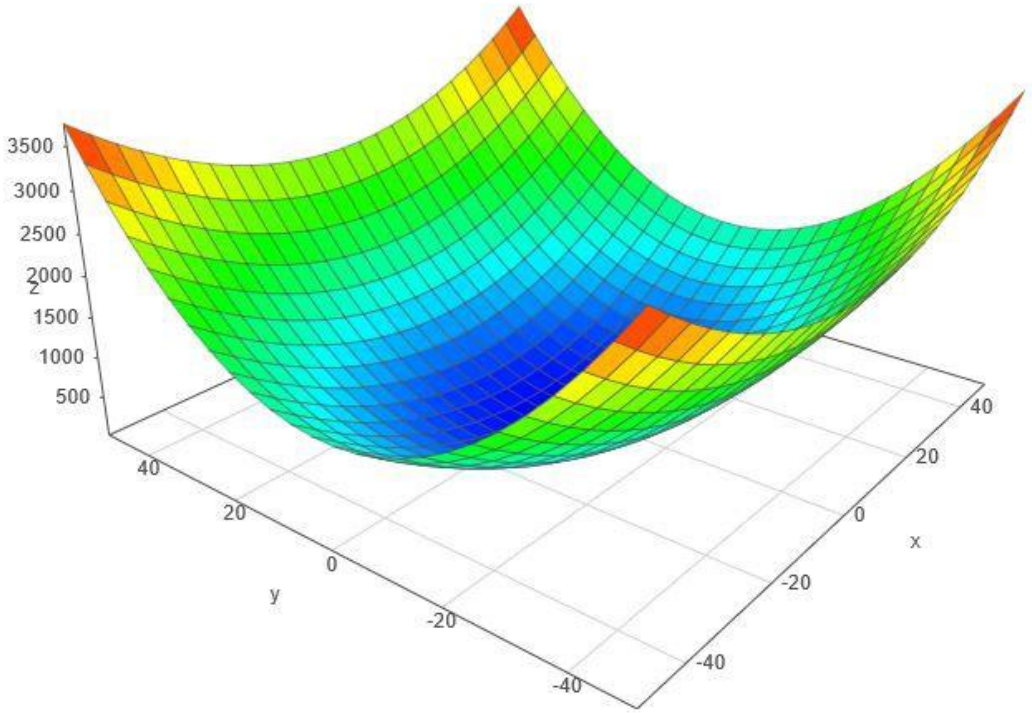
$$f(x) = 0.5x^2 + y^2$$

- gradient at point p(10,10)?

$$\frac{\partial f(x,y)}{\partial x} = x, \quad \frac{\partial f(x,y)}{\partial y} = 2y$$

$$\nabla f(x,y) = \begin{bmatrix} x \\ 2y \end{bmatrix}$$

$$\nabla f(10,10) = \begin{bmatrix} 10 \\ 20 \end{bmatrix}$$



- the slope is twice steeper along the y axis.

- Gradient Descent Algorithm iteratively calculates the next point using gradient at the current position, scales it (by a learning rate) and subtracts obtained value from the current position (makes a step). It subtracts the value because we want to minimize the function (to maximize it would be adding). This process can be written as:

$$p_{n+1} = p_n - \eta \nabla f(p_n)$$

- There's an important parameter η which scales the gradient and thus controls the step size. In machine learning, it is called **learning rate** and have a strong influence on performance.
  - The smaller learning rate the longer GD converges, or may reach maximum iteration before reaching the optimum point
  - If learning rate is too big the algorithm may not converge to the optimal point (jump around) or even to diverge completely.

STEPS

1. choose a starting point (initialization)
2. calculate gradient at this point
3. make a scaled step in the opposite direction to the gradient (objective: minimize)
4. repeat points 2 and 3 until one of the criteria is met:
   - maximum number of iterations reached
   - step size is smaller than the tolerance (due to scaling or a small gradient).

$$f(x) = x^2 - 4x + 1$$

$$\frac{df(x)}{dx} = 2x - 4$$

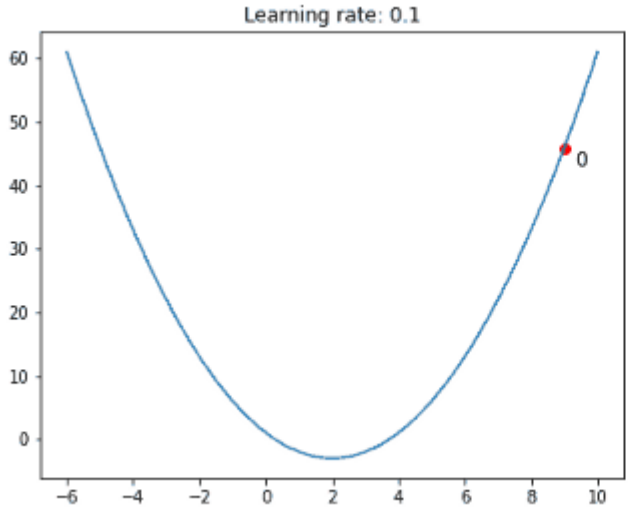
history, result = gradient_descent(9, gradient_func1, 0.1, 100)

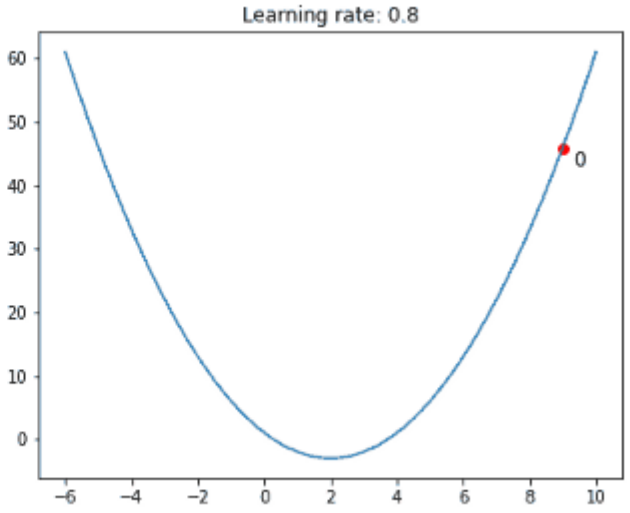
```python
import numpy as np

def gradient_descent(start, gradient, learn_rate, max_iter, tol=0.01):
  steps = [start] # history tracking
  x = start

  for _ in range(max_iter):
    diff = learn_rate*gradient(x)
    if np.abs(diff)<tol:
      break
    x = x - diff
    steps.append(x) # history tracing

  return steps, x
```

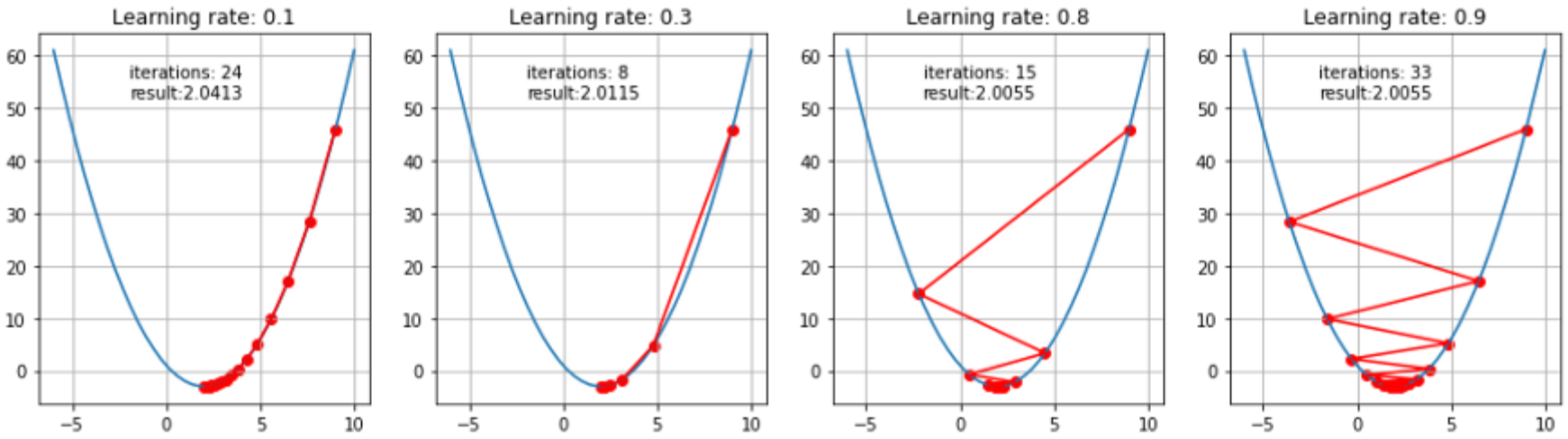
```python
def func1(x):
  return x**2-4*x+1

def gradient_func1(x):
  return 2*x - 4
```

START X=9, LEARNING = 0.1

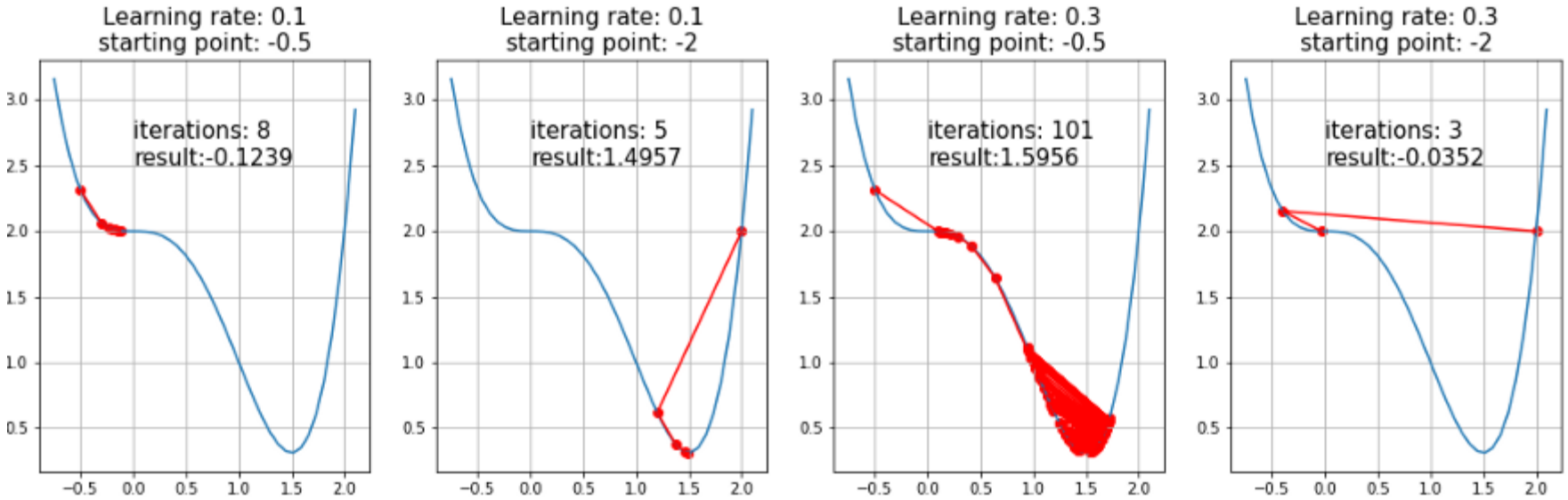
$$x_1 = 9 - 0.1 \cdot (2 \cdot 9 - 4) = 7.6$$
$$x_2 = 7.6 - 0.1 \cdot (2 \cdot 7.6 - 4) = 6.48$$
$$x_3 = 6.48 - 0.1 \cdot (2 \cdot 6.48 - 4) = 5.584$$

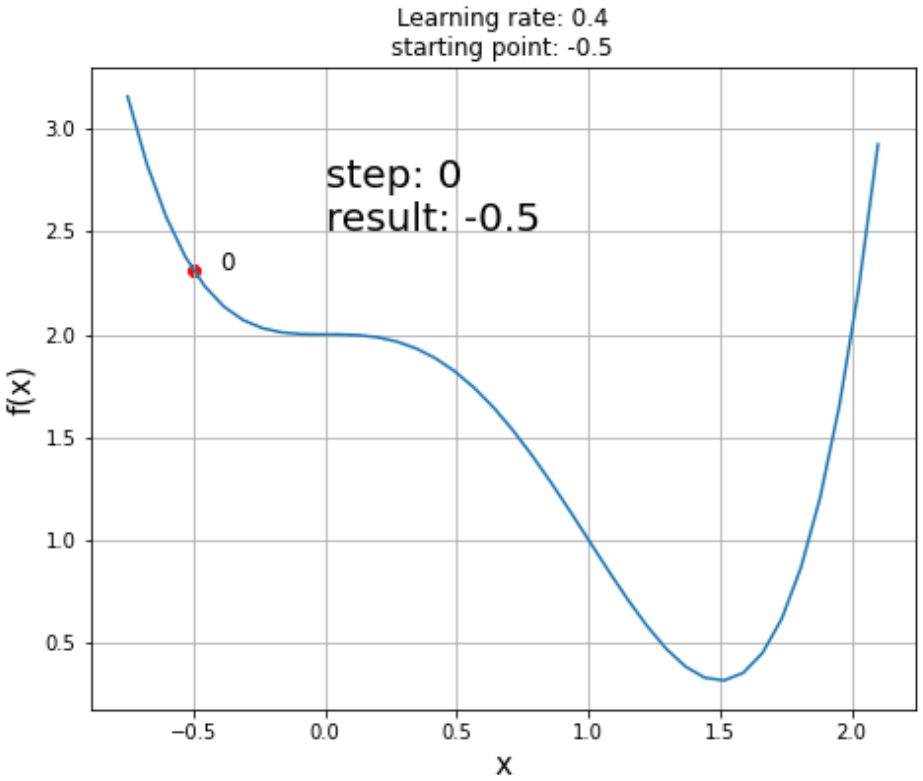





Learning rate: 0.1
iterations: 24
result:2.0413
Learning rate: 0.3
iterations: 8
result:2.0115
Learning rate: 0.8
iterations: 15
result:2.0055
Learning rate: 0.9
iterations: 33
result:2.0055

$$f(x) = x^4 - 2x^3 + 2$$


Learning rate: 0.1
starting point: -0.5
iterations: 8
result:-0.1239
Learning rate: 0.1
starting point: -2
iterations: 5
result:1.4957
Learning rate: 0.3
starting point: -0.5
iterations: 101
result:1.5956
Learning rate: 0.3
starting point: -2
iterations: 3
result:-0.0352


Learning rate: 0.4
starting point: -0.5
step: 0
result: -0.5
0
f(x)
x
3.0
2.5
2.0
1.5
1.0
0.5
−0.5
0.0
0.5
1.0
1.5
2.0

PYTORCH – QUICK REFERENCE

- Data Manipulation
- Data Preprocessing
- Linear Algebra
- Calculus
- Automatic Differentiation
- Probability and Statistics

TENSOR

- A tensor represents a (possibly multi-dimensional) array of numerical values. With one axis, a tensor is called a ***vector***. With two axes, a tensor is called a ***matrix***. With k > 2 axes, we drop the specialized names and just refer to the object as a ***kth order*** tensor

- `x = torch.arange(12, dtype=torch.float32)`
- `x`
- `tensor([ 0., 1., 2., 3., 4., 5., 6., 7., 8., 9., 10., 11.])`
- `X = x.reshape(3, 4)` ???
- `torch.zeros((2, 3, 4))` ???
- `torch.randn(3, 4)`
- `torch.tensor([[2, 1, 4, 3], [1, 2, 3, 4], [4, 3, 2, 1]])`

DATA MANIPULATION

- As with Python lists, we can access tensor elements by indexing (starting with 0). To access an element based on its position relative to the end of the list, we can use negative indexing. Finally, we can access whole ranges of indices via slicing (e.g., X[start:stop]), where the returned value includes the first index (start) but not the last (stop). Finally, when only one index (or slice) is specified for a order tensor, it is applied along axis 0. Thus, in the following code, [-1] selects the last row and [1:3] selects the second and third rows.

- X[-1], X[1:3]

- If we want to assign multiple elements the same value, we apply the indexing on the left-hand side of the assignment operation. For instance, [:2, :] accesses the first and second rows, where : takes all the elements along axis 1 (column). While we discussed indexing for matrices, this also works for vectors and for tensors of more than 2 dimensions.
- X[:2, :] = 12

OPERATIONS

- Apply a standard scalar operation to each element of a tensor. For functions that take two tensors as inputs, elementwise operations apply some standard binary operator on each pair of corresponding elements

- torch.exp(x)

- x = torch.tensor([1.0, 2, 4, 8])
- y = torch.tensor([2, 2, 2, 2])
- x + y, x - y, x * y, x / y, x ** y

TRY

- X = torch.arange(12, dtype=torch.float32).reshape((3,4))
- Y = torch.tensor([[2.0, 1, 4, 3], [1, 2, 3, 4], [4, 3, 2, 1]])
- torch.cat((X, Y), dim=0), torch.cat((X, Y), dim=1)

```
(tensor([[ 0.,  1.,  2.,  3.],
         [ 4.,  5.,  6.,  7.],
         [ 8.,  9., 10., 11.],
         [ 2.,  1.,  4.,  3.],
         [ 1.,  2.,  3.,  4.],
         [ 4.,  3.,  2.,  1.]]),
 tensor([[ 0.,  1.,  2.,  3.,  2.,  1.,  4.,  3.],
         [ 4.,  5.,  6.,  7.,  1.,  2.,  3.,  4.],
         [ 8.,  9., 10., 11.,  4.,  3.,  2.,  1.]]))
```

- X == Y ???
- X.sum() ???

BROADCASTING

- a = torch.arange(3).reshape((3, 1))
- b = torch.arange(2).reshape((1, 2))
- a, b

- Broadcasting produces a larger matrix by replicating matrix a along the columns and matrix b along the rows before adding them elementwise.
- a + b

DATA PROCESSING

```
import os

os.makedirs(os.path.join('..', 'data'), exist_ok=True)
data_file = os.path.join('..', 'data', 'house_tiny.csv')
with open(data_file, 'w') as f:
    f.write('''NumRooms,RoofType,Price
NA,NA,127500
2,NA,106000
4,Slate,178100
NA,NA,140000''')
```

```
import pandas as pd

data = pd.read_csv(data_file)
print(data)
```

$$\frac{\partial y}{\partial x_i} = \lim_{h \to 0} \frac{f(x_1, \ldots, x_{i-1}, x_i + h, x_{i+1}, \ldots, x_n) - f(x_1, \ldots, x_i, \ldots, x_n)}{h}.$$

CALCULUS

$$f'(x) = \lim_{h \to 0} \frac{f(x+h) - f(x)}{h}.$$

$$\frac{d}{dx} C = 0 \qquad \text{for any constant } C$$

$$\frac{d}{dx} x^n = n x^{n-1} \qquad \text{for } n \neq 0$$

$$\frac{d}{dx} e^x = e^x$$

$$\frac{d}{dx} \ln x = x^{-1}$$

$$\frac{dy}{dx} = \frac{dy}{du} \frac{du}{dx}.$$

$$\frac{d}{dx}[C f(x)] = C \frac{d}{dx} f(x) \qquad \text{Constant multiple rule}$$

$$\frac{d}{dx}[f(x) + g(x)] = \frac{d}{dx} f(x) + \frac{d}{dx} g(x) \qquad \text{Sum rule}$$

$$\frac{d}{dx}[f(x) g(x)] = f(x) \frac{d}{dx} g(x) + g(x) \frac{d}{dx} f(x) \qquad \text{Product rule}$$

$$\frac{d}{dx} \frac{f(x)}{g(x)} = \frac{g(x) \frac{d}{dx} f(x) - f(x) \frac{d}{dx} g(x)}{g^2(x)} \qquad \text{Quotient rule}$$

AUTOMATIC DIFFERENTIATION

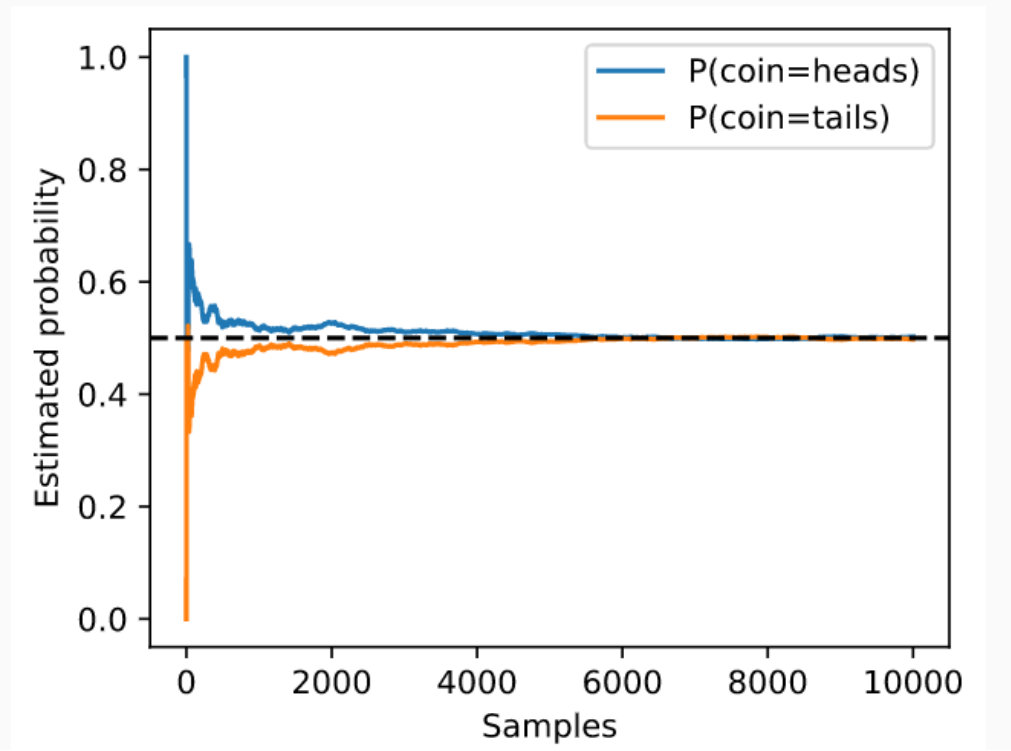
- Differentiate $y = 2x^T x$ to the column vector x ($y = 2x^2$)
- x = torch.arange(4.0)

- x.requires_grad_(True)
- x.grad # The default value is None
- y = 2 * torch.dot(x, x)
  - tensor(28., grad_fn=<MulBackward0>)
- y.backward()
- x.grad
  - tensor([0., 4., 8., 12.])
- x.grad == 4 * x
  - tensor([True, True, True, True])

counts = Multinomial(10000, fair_probs).sample()
counts / 10000

PROBABILITY AND STATISTICS

- %matplotlib inline
- import random
- import torch
- from torch.distributions.multinomial import Multinomial
- from d2l import torch as d2l
- num_tosses = 100
- heads = sum([random.random() > 0.5 for _ in range(100)])
- tails = num_tosses - heads
- print("heads, tails: ", [heads, tails])

- counts = Multinomial(1, fair_probs).sample((10000,))
- cum_counts = counts.cumsum(dim=0)
- estimates = cum_counts / cum_counts.sum(dim=1, keepdims=True)
- estimates = estimates.numpy()

- d2l.set_figsize((4.5, 3.5))
- d2l.plt.plot(estimates[:, 0], label=("P(coin=heads)"))
- d2l.plt.plot(estimates[:, 1], label=("P(coin=tails)"))
- d2l.plt.axhline(y=0.5, color='black', linestyle='dashed')
- d2l.plt.gca().set_xlabel('Samples')
- d2l.plt.gca().set_ylabel('Estimated probability')
- d2l.plt.legend();